

Core Banking & Payment Gateway

Tài liệu học v2 cho Backend Developer: hiểu banking domain, đọc kiến trúc core banking, nắm flow nghiệp vụ và tích hợp an toàn với Core/T24.

Phiên bản chỉnh sửa: format hiện đại + sơ đồ kiến trúc + Mermaid source + use case thực tế

Đối tượng

Backend Java Developer, Integration Developer, System Analyst mới vào banking domain.

Phạm vi

Core Banking, ESB, API Gateway, Deposit, Accounting, Payment Gateway, NAPAS/VNPAY, Reliability patterns.

Ngày biên soạn

30/06/2026

Cách học

Đọc theo flow: Domain → Architecture → Transaction → Accounting → Integration → Production.

Mục lục

Tài liệu này được viết lại từ file gốc theo hướng dễ học hơn: khái niệm nào là thuật ngữ riêng đều được giải thích, mỗi kiến trúc có sơ đồ, mỗi flow có điểm cần chú ý khi làm backend.

01 - Cách đọc tài liệu và mental model tổng quan

03 - Banking domain foundation: CIF, Account, Product, Balance

05 - Deposit module: mở tài khoản, chuyển tiền, tính lãi

07 - ESB, API Gateway, Canonical Model và Message Format

09 - Use case thực tế hay gặp trong ngân hàng

11 - Data architecture: OLTP, ledger, DWH, locking, audit

13 - Batch/EOD, reconciliation và vận hành production

15 - Glossary thuật ngữ Core Banking

02 - Bức tranh kiến trúc Core Banking + Payment Gateway

04 - Transaction, Posting, Journal, Ledger và Accounting

06 - Loan, Card, FX, Treasury - hiểu đủ để đọc hệ thống

08 - Payment Gateway, NAPAS, VNPAY và vòng đời thanh toán

10 - Reliability patterns: idempotency, timeout, reversal, outbox

12 - Security architecture: mTLS, HSM, masking, maker-checker

14 - Checklist thiết kế API banking cho backend

16 - Phụ lục Mermaid source và nguồn tham khảo

1. Cách đọc tài liệu và mental model tổng quan

Mục tiêu của tài liệu không phải biến bạn thành core banking consultant trong vài ngày. Mục tiêu thực tế hơn là giúp backend developer hiểu đủ banking domain, đọc được flow T24/Core Banking, biết tích hợp an toàn, tránh các lỗi nguy hiểm như double debit, timeout không rõ trạng thái, sai hạch toán, log lộ dữ liệu nhạy cảm.

01

Hiểu domain: CIF, account, balance, deposit, loan, card, GL.

02

Hiểu architecture: channel, gateway, ESB, core, database, external network.

03

Biết tích hợp an toàn: idempotency, query status, reversal, reconciliation, audit.

1.1 Một câu nhớ toàn bộ hệ thống

Mental model quan trọng

Channel nhận request, API Gateway kiểm soát lối vào, Digital Domain xử lý trải nghiệm kênh, ESB điều phối và chuyển đổi message, Core Banking xử lý tiền thật, Accounting ghi sổ, Database lưu trạng thái, Payment Network kết nối bên ngoài.

1.2 Tại sao banking khó hơn web CRUD?

- **Tiền không được tự sinh ra hoặc mất đi.** Mọi giao dịch tài chính phải cân bằng debit = credit.
- **Không được ghi trùng.** User bấm hai lần, app retry, network timeout đều có thể gây double debit.
- **Timeout không đồng nghĩa thất bại.** Core có thể đã commit nhưng response bị mất.
- **Audit là bắt buộc.** Cần biết ai làm, lúc nào, kênh nào, traceId nào, request/response gì.
- **Batch cuối ngày rất quan trọng.** Lãi, phí, GL, report, reconciliation thường chạy theo business date.
- **Tích hợp legacy nhiều.** Không phải hệ thống nào cũng REST JSON; có fixed-length, ISO 8583, ISO 20022, MQ, file, TCP socket.
- **Security/compliance cao.** Cần masking, encryption, HSM, mTLS, maker-checker, segregation of duties.

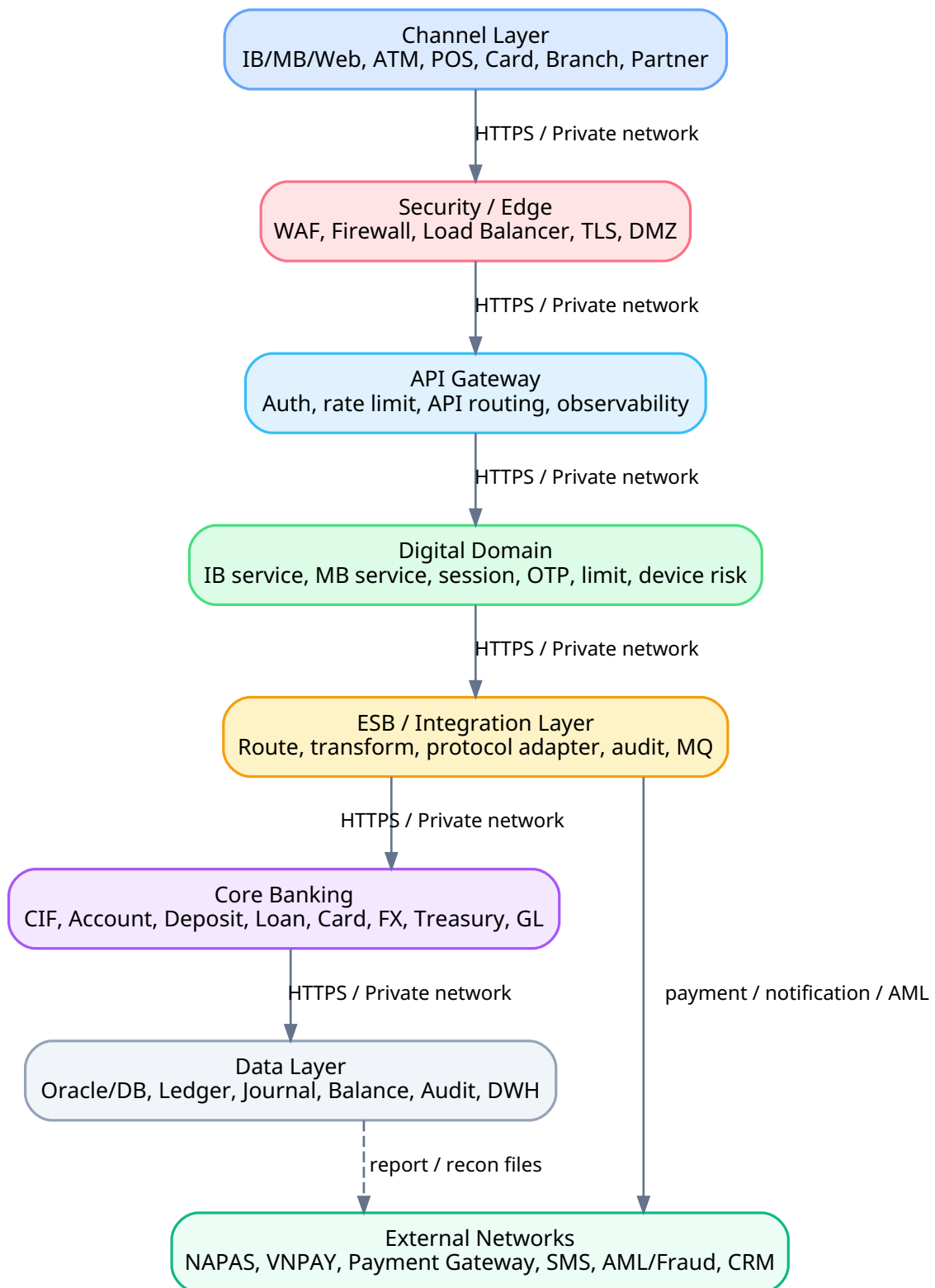
1.3 Thứ tự học khuyến nghị

1. Bắt đầu bằng khái niệm domain: Customer/CIF, Account, Product, Balance.
2. Hiểu transaction khác posting như thế nào; journal khác ledger như thế nào.
3. Đi vào Deposit vì đây là module gần nhất với account, balance, transfer và lãi tiền gửi.
4. Sau đó học Payment Gateway và các khái niệm authorization, clearing, settlement, reconciliation.
5. Cuối cùng học reliability patterns và production concerns: idempotency, timeout, reversal, locking, audit, EOD.

2. Bức tranh kiến trúc Core Banking + Payment Gateway

Một hệ thống ngân hàng hiện đại thường có nhiều kênh giao dịch: Internet Banking, Mobile Banking, ATM, POS, Card, Branch/Teller, Call Center và Partner API. Các kênh này không nên gọi trực tiếp vào Core Banking vì Core là vùng nhạy cảm nhất, cần được bảo vệ bằng nhiều lớp security, integration, audit và kiểm soát trạng thái.

Sơ đồ 1 - Kiến trúc tổng quan Core Banking + Payment Gateway



Luồng đi từ channel vào core và các hệ thống ngoài như NAPAS, VNPAY, SMS, AML/Fraud, CRM.

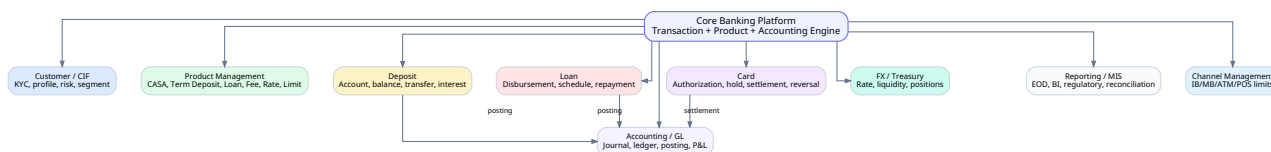
2.1 Vai trò từng layer

Layer	Vai trò	Lỗi/rủi ro cần kiểm soát
Channel Layer	Web/Mobile/ATM/POS/Branch/Partner là nơi phát sinh request.	Session hết hạn, user bấm lặp, thiết bị rủi ro, request giả mạo.
Security / Edge	Firewall, WAF, Load Balancer, TLS termination, DMZ.	Tấn công web, DDoS, truy cập trái phép, cấu hình TLS yếu.
API Gateway	Expose API, auth, rate limit, routing, logging, monitoring.	Lộ API nội bộ, thiếu rate limit, không correlate log.
Digital Domain	Backend riêng của IB/MB/Card: session, OTP, device risk, limit kênh.	Validate thiếu, retry mù, lưu token/OTP sai cách.
ESB / Integration	Route, transform, protocol adapter, MQ, audit, mapping core.	Map sai field, timeout, bottleneck, retry không kiểm soát.
Core Banking	Nơi xử lý account, deposit, loan, card, posting, accounting.	Sai balance, double posting, deadlock, GL lệch.
Data / Reporting	DB, ledger, journal, audit, DWH, BI, reconciliation.	Replica lag, report làm chậm OLTP, mất audit trail.
External Networks	NAPAS, VNPAY, Payment Gateway, SMS, AML/ Fraud, CRM.	Callback giả, settlement lệch, đối soát sai, network timeout.

2.2 Core Banking là gì?

Core Banking là hệ thống lõi quản lý khách hàng, tài khoản, sổ dư, sản phẩm tài chính, giao dịch, hạch toán và báo cáo vận hành. Với mobile/web system bình thường, dữ liệu sai có thể sửa tương đối dễ. Với Core Banking, sai sổ dư hoặc sai hạch toán có thể ảnh hưởng đến tiền thật, báo cáo tài chính, đối soát và pháp lý.

Sơ đồ 2 - Các module chính trong Core Banking



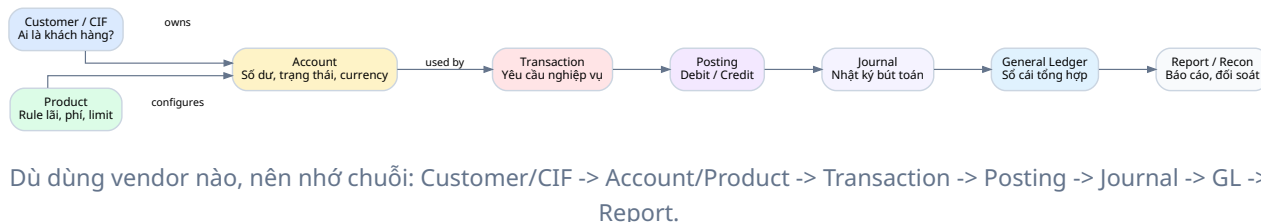
Core Banking không chỉ có account; nó gồm product, deposit, loan, card, FX/treasury, accounting/GL và reporting.

2.3 Core Banking vendor/platform thường gặp

Platform	Cách hiểu	Điểm cần học
Temenos T24 / Temenos Transact	Core banking platform phổ biến. Temenos gọi Arrangement Architecture là framework để tạo/quản lý products như accounts, deposits, loans theo kiến trúc component-based.	Customer, Account, Funds Transfer, AA Product, AA Arrangement, OFS/API, COB/EOD.
Oracle FLEXCUBE	Core banking enterprise của Oracle, tập trung vào product management, operation và connectivity.	Product, account, GL, transaction processing, integration.
Thought Machine Vault Core	Modern core cloud-native/API-first, product logic qua smart contracts/product engine.	Ledger, product engine, API-first, event-driven thinking.
In-house Core	Ngân hàng tự xây core hoặc các module lõi riêng.	Domain model, consistency, accounting, audit, batch, reconciliation.

3. Banking domain foundation: CIF, Account, Product, Balance

Sơ đồ 3 - Xương sống domain trong Core Banking



3.1 CIF là gì?

CIF thường được hiểu là Customer Information File: hồ sơ định danh khách hàng trong ngân hàng. CIF không phải là tài khoản. Một khách hàng có một hoặc nhiều CIF tùy ngân hàng, và một CIF có thể sở hữu nhiều tài khoản/sản phẩm.

Thông tin trong CIF	Ý nghĩa
Tên, ngày sinh/ngày thành lập	Xác định cá nhân/doanh nghiệp.
Giấy tờ định danh: CCCD, passport, business registration	Phục vụ KYC và tuân thủ.
KYC status, AML risk rating	Đánh giá khách hàng đã xác minh chưa, có rủi ro không.
Segment: retail, SME, corporate, VIP	Ảnh hưởng phí, hạn mức, sản phẩm, chăm sóc khách hàng.
Relationship manager / branch	Ai quản lý khách hàng, thuộc chi nhánh nào.

Ví dụ

```
CIF: CIF00012345
Name: Nguyen Van A
KYC Status: VERIFIED
Risk Rating: LOW
Segment: Retail
Accounts:
- CASA account: 00110001111
- Term deposit: 00330003333
- Loan account: 00990009999
- Card: 9704 xxxx xxxx 1234
```


3.2 Account là gì?

Account là tài khoản dùng để ghi nhận số dư hoặc nghĩa vụ tài chính. Trong ngân hàng, account không chỉ là tài khoản thanh toán của khách hàng. Nó còn có loan account, internal account, GL account, nostro/vostro account.

Loại account	Giải thích	Ví dụ
Customer account	Tài khoản của khách hàng để nạp/rút/chuyển tiền.	CASA, current account, savings account.
Loan account	Tài khoản ghi nhận khoản vay và dư nợ.	Loạn mua nhà, loan mua xe.
Internal account	Tài khoản nội bộ ngân hàng dùng cho nghiệp vụ trung gian.	Suspense account, clearing account.
GL account	Tài khoản kế toán tổng hợp.	Cash, Customer Deposit Liability, Interest Income.
Nostro/Vostro	Tài khoản ngân hàng dùng trong thanh toán liên ngân hàng/quốc tế.	Ngân hàng A mở tài khoản tại ngân hàng B.

3.3 Product là gì?

Trong banking, product không chỉ là tên marketing. Product là tập hợp rule vận hành: loại tài khoản, tiền tệ, lãi suất, cách tính lãi, phí, hạn mức, điều kiện đóng/mở, GL mapping và rule hạch toán.

Product vs Account

Product là mẫu/rule sản phẩm. **Account** là tài khoản cụ thể của khách hàng được tạo theo product đó. Với T24 hiện đại, **Arrangement** có thể được hiểu như hợp đồng/sản phẩm cụ thể của khách hàng trên Arrangement Architecture.

3.4 Balance là gì?

Balance không chỉ có một loại. Backend developer làm banking phải biết dùng loại balance nào cho từng quyết định nghiệp vụ.

Loại balance	Ý nghĩa	Dùng khi nào
Ledger balance	Số dư đã ghi sổ trong core/ledger.	Dùng cho báo cáo, sao kê, đối chiếu kế toán.
Available balance	Số dư khách có thể dùng sau khi trừ hold/block/minimum balance.	Dùng để quyết định có cho chuyển/rút tiền không.
Current balance	Cách gọi tùy hệ thống; có thể gần với ledger hoặc real-time balance.	Hiển thị app, nhưng phải hiểu rõ định nghĩa của bank.
Hold amount	Số tiền bị giữ tạm, chưa được dùng.	Card authorization, giao dịch đang pending, dispute.
Blocked/Frozen amount	Số tiền bị phong tỏa do pháp lý/rủi ro/quy trình.	Không cho giao dịch dù ledger balance còn tiền.

```
Ledger balance    = 10,000,000 VND
Hold amount       = 2,000,000 VND
Minimum balance   =   50,000 VND
Available balance =  7,950,000 VND
```

Kết luận: không được check đơn giản `balance > amount`.
Phải check đúng rule: `available balance`, `hold`, `freeze`, `limit`, `product`, `channel`.

4. Transaction, Posting, Journal, Ledger và Accounting

4.1 Transaction là gì?

Transaction là yêu cầu nghiệp vụ làm thay đổi trạng thái tài chính hoặc trạng thái vận hành. Ví dụ: chuyển tiền, nộp tiền, rút tiền, mở tài khoản, tính lãi, thu phí, giải ngân, trả nợ, reversal, refund.

Field nên có	Ý nghĩa
transaction_id	ID nội bộ của hệ thống.
request_id	ID từ channel/client, dùng cho idempotency.
trace_id / correlation_id	ID trace xuyên hệ thống.
channel	IB, MB, ATM, POS, teller, batch, partner.
service_code	Loại nghiệp vụ: FUND_TRANSFER, BALANCE_INQUIRY, BILL_PAYMENT.
amount/currency	Số tiền và loại tiền.
debit_account / credit_account	Tài khoản bị ghi nợ / ghi có.
status	INIT, VALIDATED, POSTING, SUCCESS, FAILED, PENDING, REVERSED.
response_code	Mã phản hồi core/payment network.
created_at / posted_at	Thời điểm nhận request và thời điểm hạch toán.

4.2 Posting là gì?

Posting là bước biến một transaction nghiệp vụ thành các bút toán debit/credit. Đây là điểm khác biệt lớn giữa banking và CRUD system. Một request chuyển tiền không chỉ update số dư; nó phải tạo bút toán cân bằng để kế toán và ledger đúng.

Sơ đồ 4 - Luồng posting và accounting trong giao dịch tiền

```
graph LR; 1[1. Receive transfer requestid + traceid] --> 2[2. Idempotency check unique(channel, requestid)]; 2 --> 3[3. Validate account, OTP, limit, status]; 3 --> 4[4. Lock balance SELECT FOR UPDATE / version]; 4 --> 5[5. Posting Debit A, Credit B, fee/VAT]; 5 --> 6[6. Journal + Ledger append-only lines]; 6 --> 7[7. Commit DB mark SUCCESS]; 7 --> 8[8. Outbox events notification, DWH, fraud];
```

Transaction sau khi validate sẽ lock balance, tạo posting, journal lines, cập nhật ledger/balance, audit và report.

Nguyên tắc vàng

Tổng debit phải bằng tổng credit. Nếu không cân bằng, transaction phải rollback hoặc vào queue xử lý lỗi có kiểm soát.

4.3 Journal và General Ledger

Journal là nhật ký bút toán theo thời gian, thường có header và các line debit/credit. **General Ledger (GL)** là sổ cái tổng hợp theo tài khoản kế toán. Journal chi tiết hơn; GL dùng cho tổng hợp báo cáo tài chính.

Khái niệm	Mục đích	Ví dụ
Journal Entry	Header của bút toán: transaction_id, posting_date, value_date, branch, channel.	JE-20260630-0001
Journal Line	Dòng debit/credit trong journal entry.	Debit Cash 10m, Credit Customer Deposit 10m.
GL Account	Tài khoản kế toán tổng hợp.	101000 Cash, 201000 Customer Deposit Liability.
Sub-ledger	Sổ chi tiết theo customer/account/product.	Chi tiết balance từng tài khoản khách hàng.
P&L	Profit & Loss, báo cáo thu nhập/chi phí trong kỳ.	Interest income, fee income, interest expense.
Balance Sheet	Bức tranh tài sản/nợ/vốn tại một thời điểm.	Assets = Liabilities + Equity.

4.4 Debit/Credit theo góc nhìn ngân hàng

Điểm dễ nhầm nhất: tiền trong tài khoản khách hàng là tài sản của khách hàng, nhưng là nợ phải trả của ngân hàng. Khi khách gửi tiền, ngân hàng có thêm asset nhưng đồng thời tăng liability với khách hàng.

Loại tài khoản	Tăng bằng	Giảm bằng	Banking example
Asset	Debit	Credit	Cash, Nostro, Loan Asset.
Liability	Credit	Debit	Customer Deposit Liability.
Equity	Credit	Debit	Vốn chủ sở hữu.
Income	Credit	Debit	Interest Income, Fee Income.
Expense	Debit	Credit	Interest Expense, Operating Expense.

4.5 Ví dụ hạch toán thực tế

Case	Debit	Credit	Ý nghĩa
Khách nộp 10 triệu	Cash / Nostro Asset +10,000,000	Customer Deposit Liability +10,000,000	Ngân hàng có thêm tiền và nợ khách thêm tiền.
A chuyển B cùng ngân hàng	Deposit Liability - Account A 1,000,000	Deposit Liability - Account B 1,000,000	Giảm nghĩa vụ với A, tăng nghĩa vụ với B.
Giải ngân loan 100 triệu	Loan Asset +100,000,000	Customer Deposit Liability +100,000,000	Ngân hàng có khoản phải thu và ghi có vào tài khoản khách.
Tính lãi tiền gửi	Interest Expense	Interest Payable / Customer Deposit	Lãi tiền gửi là chi phí của ngân hàng.
Thu lãi vay	Customer Deposit Liability / Cash	Interest Income	Lãi vay là thu nhập của ngân hàng.

5. Deposit module: mở tài khoản, chuyển tiền, tính lãi

5.1 Deposit là gì?

Deposit là module quản lý tiền gửi của khách hàng: tài khoản thanh toán, tiết kiệm không kỳ hạn, tiền gửi có kỳ hạn, gửi góp, flexi deposit. Trong đoạn training gốc, Deposit được mô tả là xử lý mở tài khoản, chuyển tiền, tính lãi, quản lý balance và ghi nhận ledger/account; đây là cách hiểu high-level đúng.

Loại deposit	Tên đầy đủ	Ý nghĩa
CASA	Current Account / Savings Account	Tài khoản thanh toán/tiết kiệm không kỳ hạn.
Current Account	Tài khoản vãng lai	Thường dùng cho doanh nghiệp, giao dịch nhiều, lãi thấp hoặc không lãi.
Savings Account	Tiết kiệm không kỳ hạn	Dùng cho cá nhân, lãi thấp, rút linh hoạt.
Term Deposit	Tiền gửi có kỳ hạn	Gửi cố định 1/3/6/12 tháng, lãi suất cao hơn.
Recurring Deposit	Gửi góp định kỳ	Khách hàng nộp tiền định kỳ theo lịch.
Flexi Deposit	Tiền gửi linh hoạt	Kết hợp thanh toán và tiết kiệm, có rule sweep tự động.

5.2 Luồng mở tài khoản deposit

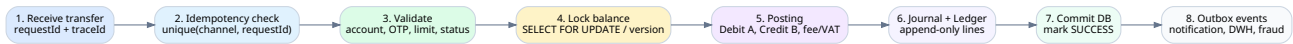
1. Khách hàng đăng ký qua IB/MB hoặc tại quầy.
2. Channel backend kiểm tra session, OTP, consent nếu cần.
3. Core kiểm tra CIF đã tồn tại chưa.
4. Nếu chưa có CIF: tạo CIF sau khi KYC hợp lệ.
5. Chọn product: CASA, Savings, Term Deposit.
6. Core tạo account number, currency, branch, status.
7. Khởi tạo balance và gán rule interest/fee/limit.
8. Gắn GL mapping theo product.
9. Nếu có tiền nạp ban đầu: tạo posting debit/credit.
10. Trả account_no và trạng thái về channel.

```
{
  "requestId": "REQ-20260630-0001",
  "cif": "CIF00012345",
  "accountNo": "00110001111",
  "productCode": "CASA-VND-001",
  "status": "ACTIVE",
  "currency": "VND"
}
```

5.3 Luồng chuyển tiền cùng ngân hàng

Chuyển tiền cùng ngân hàng thường có thể được xử lý atomic trong core vì cả tài khoản nguồn và đích nằm trong cùng hệ thống. Tuy nhiên vẫn cần idempotency, locking, audit và posting cân bằng.

Sơ đồ 5 - Flow chuyển tiền nội bộ an toàn



Flow backend nên đi qua idempotency check, validate, lock balance, posting, journal, commit và outbox event.

5.4 Interest calculation trong Deposit

Lãi tiền gửi có thể tính theo ngày, tháng hoặc kỳ hạn. Công thức đơn giản chỉ để học khái niệm; thực tế còn day-basis, lãi suất bậc thang, rounding, tax, early withdrawal, capitalization và accrual.

```
interest = principal * annual_rate * number_of_days / day_basis
```

Example:

principal = 100,000,000 VND

annual_rate = 5% = 0.05

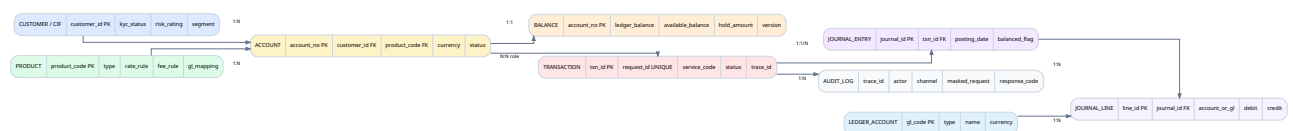
number_of_days = 30

day_basis = 365

interest = 100,000,000 * 0.05 * 30 / 365 = 410,958 VND

5.5 Data model logic của Deposit

Sơ đồ 6 - Data model logic tối giản



Các bảng logic thường gặp: customer, product, account, balance, transaction, journal, journal line, ledger account, audit log.

Bảng logic	Vai trò
customer / cif	Thông tin khách hàng, KYC, segment, risk.
product	Loại sản phẩm deposit, rule lãi/phí/limit/GL.
account	Tài khoản khách hàng và trạng thái.
balance	Ledger balance, available balance, hold amount, version.
transaction	Giao dịch phát sinh và trạng thái.
journal_entry	Header bút toán.
journal_line	Dòng debit/credit.
ledger_account	Tài khoản kế toán tổng hợp.
audit_log	Log phục vụ audit/trace/debug.

6. Loan, Card, FX, Treasury - hiểu đủ để đọc hệ thống

6.1 Loan module

Loan quản lý vòng đời khoản vay: application, credit assessment, approval, contract, disbursement, repayment schedule, interest calculation, collection, overdue, closure. Với ngân hàng, loan là asset vì khách hàng nợ tiền ngân hàng.

Thuật ngữ	Giải thích
Principal	Gốc vay.
Interest	Lãi vay.
Outstanding balance	Dư nợ còn lại.
Installment	Số tiền phải trả mỗi kỳ.
Collateral	Tài sản đảm bảo.
Overdue	Quá hạn.
NPL	Non-performing loan, nợ xấu.
Provisioning	Trích lập dự phòng rủi ro.

6.2 Card module

Card module xử lý phát hành thẻ, active thẻ, PIN, limit, authorization, ATM/POS/e-commerce transaction, hold amount, clearing, settlement, reversal, dispute/chargeback. Với card, authorization online chưa chắc là settlement cuối cùng.

Card transaction khác chuyển khoản thường

Giao dịch thẻ có thể gồm authorize trước, hold amount, clearing sau, settlement sau. Vì vậy statement, available balance và ledger balance có thể khác nhau trong một khoảng thời gian.

6.3 FX module

FX xử lý ngoại hối: tỷ giá mua/bán, multi-currency account, FX position, revaluation cuối ngày và P&L do chênh lệch tỷ giá. Ví dụ khách đổi USD sang VND: debit USD account, credit VND account, ghi nhận FX spread/income và update FX position.

6.4 Treasury module

Treasury nhìn ở cấp ngân hàng: quản lý nguồn vốn, thanh khoản, cash position, interbank lending/borrowing, bond, money market, nostro account, FX position. Nếu Deposit là tiền khách gửi, Loan là tiền khách vay, thì Treasury trả lời câu hỏi: ngân hàng có đủ thanh khoản không, nguồn vốn ở đâu, rủi ro kỳ hạn thế nào.

7. ESB, API Gateway, Canonical Model và Message Format

7.1 API Gateway là gì?

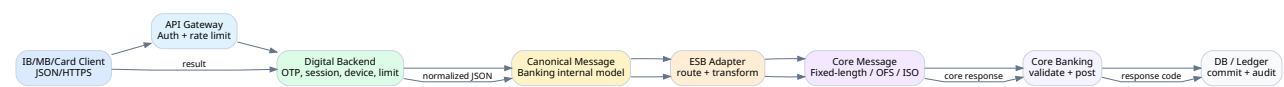
API Gateway là điểm vào API cho client hoặc system bên ngoài. Nó thường xử lý API routing, authentication/authorization, rate limiting, request validation, TLS termination, API key/JWT validation, logging/monitoring và response normalization. Trong banking, API Gateway nằm gần channel và internet-facing hơn ESB.

7.2 ESB là gì?

ESB là Enterprise Service Bus: tầng tích hợp doanh nghiệp dùng để route message, transform data model, chuyển đổi protocol, composition/orchestration, audit và kết nối nhiều hệ thống backend khác nhau. Trong banking, ESB đặc biệt quan trọng vì nhiều hệ thống core/card/payment là legacy hoặc dùng protocol/message format khác nhau.

Tiêu chí	API Gateway	ESB / Integration Layer
Vị trí	Gần client/channel.	Gần backend/internal systems.
Mục tiêu	Expose API an toàn và thống nhất.	Tích hợp nhiều hệ thống nội bộ.
Protocol	Chủ yếu HTTP/REST/gRPC.	HTTP, SOAP, MQ, TCP, file, fixed-length, ISO.
Logic chính	Auth, rate limit, routing, monitoring.	Routing, transform, orchestration, protocol adapter.
Consumer	Web, mobile, partner API.	Core, payment, card, CRM, DWH, AML.
Use case banking	Mobile app gọi API chuyển tiền.	Chuyển JSON từ MB domain thành core message.

Sơ đồ 7 - Luồng message từ Channel vào Core Banking



Message có thể đi từ JSON hiện đại qua canonical model rồi transform thành fixed-length/OFS/ISO cho core.

7.3 Canonical model là gì?

Canonical model là format trung gian chuẩn nội bộ để giảm số lượng mapping giữa các hệ thống. Thay vì IB, MB, Card, CRM, DWH map chéo trực tiếp với nhau, tất cả map về một banking message chung rồi adapter chuyển sang format đích.

Lợi ích và rủi ro của canonical model

Lợi ích: giảm mapping, dễ audit, dễ thêm channel, dễ thay backend.

Rủi ro: nếu governance kém, canonical model trở thành “God schema” quá to; ESB có thể thành bottleneck nếu nhồi quá nhiều business logic vào đó.

7.4 Vì sao có JSON -> fixed-length?

Mobile/Web hiện đại thường dùng JSON, nhưng core legacy có thể nhận fixed-length message, OFS, ISO 8583, ISO 20022 hoặc format riêng. ESB/adaptor chịu trách nhiệm mapping field, padding, validation, encoding và response code.

JSON request:

```
{
  "fromAccount": "001123456789",
  "toAccount": "001987654321",
  "amount": 1000000,
  "currency": "VND",
  "remark": "transfer"
}
```

Fixed-length message concept:

TRF0010011234567890019876543210000001000000VNDTRANSFER

Field layout:

- Transaction code: 6
- From account: 12
- To account: 12
- Amount: 13
- Currency: 3
- Remark: 20 padded spaces

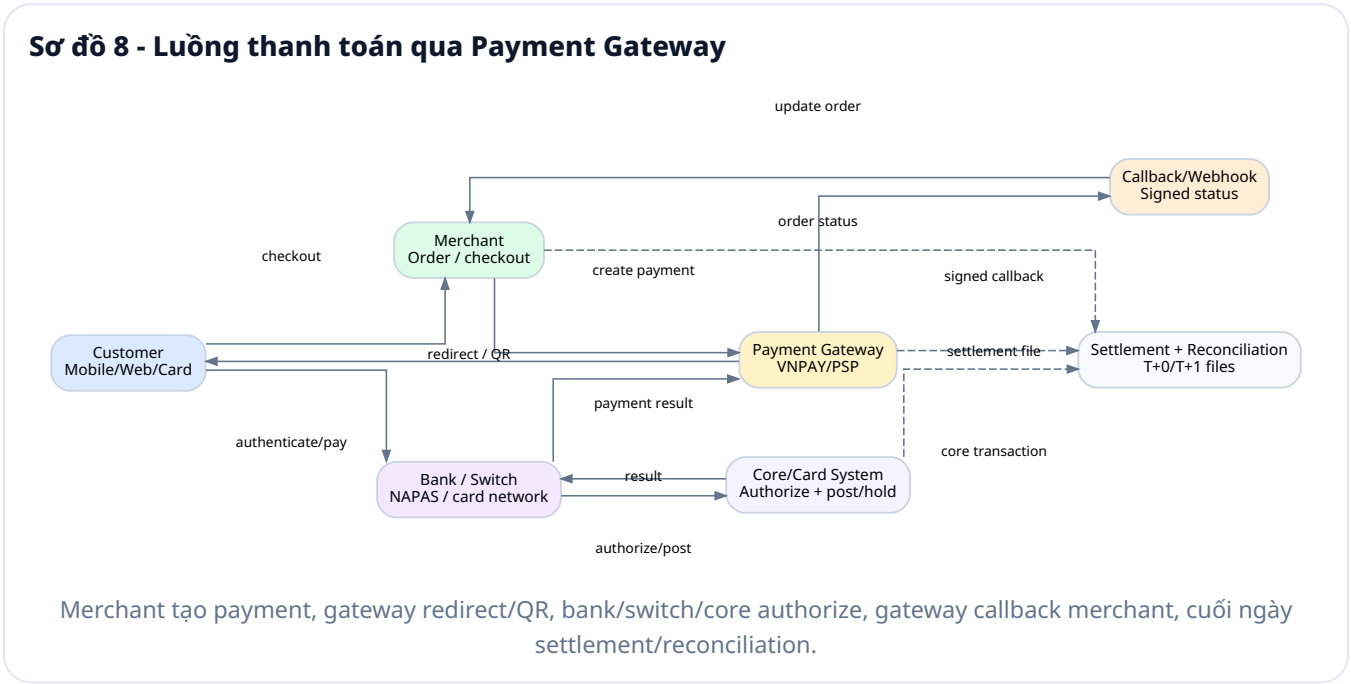
7.5 ISO 8583, ISO 20022 và OFS

Message format	Thường dùng ở đâu	Cần nhớ gì
ISO 8583	Card/ATM/POS: authorization, reversal, settlement.	Có MTI, bitmap, data elements như PAN, processing code, amount, trace number, terminal ID, response code.
ISO 20022	Payment, clearing, settlement, cash management.	Structured financial messages, giàu thông tin hơn các format cũ.
OFS	T24/Temenos legacy integration.	Có thể hiểu là message command để hệ thống ngoài yêu cầu T24 tạo/query/post nghiệp vụ.
Fixed-length	Core legacy, host system, switch.	Field có độ dài cố định; sai padding/length có thể làm core hiểu sai.
MQ/File	Batch, async integration, DWH, clearing file.	Cần retry, idempotency, file checksum, sequence, replay strategy.

8. Payment Gateway, NAPAS, VNPAY và vòng đời thanh toán

8.1 Payment Gateway là gì?

Payment Gateway là hệ thống trung gian giúp merchant nhận thanh toán từ khách hàng qua thẻ, QR, Internet Banking, Mobile Banking, ví điện tử, bank transfer hoặc payment network. Payment Gateway không giống Core Banking: nó tập trung vào merchant payment status, callback, settlement, refund và đối soát.



Tiêu chí	Core Banking	Payment Gateway
Mục tiêu	Quản lý tài khoản, sổ dư, hạch toán.	Kết nối merchant với bank/network để thanh toán.
Actor chính	Customer, account, bank branch.	Customer, merchant, PSP, bank/network.
Trạng thái quan trọng	Balance, journal, ledger.	Payment status, callback, settlement, refund.
Lỗi hay gặp	Sai balance, duplicate posting, GL lệch.	Callback giả, timeout, status mismatch, settlement lệch.

8.2 NAPAS là gì?

NAPAS có thể hiểu là hạ tầng chuyển mạch tài chính tại Việt Nam. Trong flow ATM/POS nội địa, NAPAS đóng vai trò switch giữa acquiring bank và issuing bank. Ví dụ khách dùng thẻ ngân hàng A rút tiền tại ATM ngân hàng B: request đi từ ATM B -> Bank B switch -> NAPAS -> Bank A card/core -> NAPAS -> Bank B -> ATM.

8.3 VNPAY là gì?

VNPAY là fintech/payment provider tại Việt Nam, thường xuất hiện phía Payment Gateway/PSP: kết nối merchant, QR, POS, mobile banking app, ví và ngân hàng. Trong kiến trúc, VNPAY không thay thế core banking; nó gửi/nhận payment instruction và trạng thái thanh toán với ngân hàng/network.

8.4 Authorization, Clearing, Settlement, Reconciliation

Khái niệm	Giải thích	Ví dụ
Authorization	Kiểm tra và chấp nhận giao dịch tại thời điểm online.	Quẹt thẻ 500,000; issuer kiểm tra thẻ, hạn mức, fraud, rồi approve/decline.
Clearing	Trao đổi/xác nhận dữ liệu giao dịch giữa các bên sau khi giao dịch phát sinh.	Cuối ngày acquirer gửi danh sách giao dịch POS cho network/issuer.
Settlement	Quyết toán tiền thật giữa các bên.	Issuer chuyển tiền cho acquirer/merchant sau khi trừ phí.
Reconciliation	Đối soát dữ liệu giữa merchant, gateway, bank/core, switch, GL.	Order merchant PAID nhưng core không có posting -> exception.

8.5 Callback/Webhook an toàn

- Gateway callback phải có chữ ký hoặc MAC để merchant/bank xác minh không bị giả mạo.
- Callback nên có idempotency vì gateway có thể retry nhiều lần.
- Không tin tuyệt đối vào callback nếu giao dịch có giá trị cao; cần query status hoặc đối soát.
- Không update PAID nếu amount/currency/orderId không khớp.
- Log callback phải mask PII/card data và lưu raw signature để audit.

9. Use case thực tế hay gặp trong ngân hàng

Phần này giúp bạn liên hệ kiến thức domain với công việc backend/integration thực tế. Mỗi use case đều có “điểm chết” cần kiểm soát vì nó có thể gây lệch tiền, lệch trạng thái hoặc sự cố vận hành.

ID	Use case	Mô tả	Điểm backend cần chú ý
UC01	Balance Inquiry	Query số dư từ IB/MB/ATM.	Dùng source dữ liệu đúng; tránh replica lag; mask account.
UC02	Internal Transfer	Chuyển tiền cùng ngân hàng.	Idempotency, lock balance, posting cân bằng, timeout query status.
UC03	Interbank Transfer	Chuyển tiền qua NAPAS/payment network.	Pending/unknown, clearing/settlement, reconciliation.
UC04	Merchant QR Payment	Thanh toán qua VNPAY/Payment Gateway.	Callback signature, merchant order status, refund, settlement.
UC05	ATM Withdrawal	Rút tiền qua ATM khác ngân hàng.	Authorization, dispense fail, reversal, network trace number.
UC06	Open CASA Account	Mở tài khoản thanh toán.	CIF/KYC, product rule, GL mapping, initial deposit.
UC07	Open Term Deposit	Mở sổ tiết kiệm kỳ hạn.	Maturity, interest rule, early withdrawal, renewal.
UC08	Loan Disbursement	Giải ngân khoản vay.	Loan asset, customer deposit liability, schedule, GL posting.
UC09	Loan Repayment	Trả nợ vay.	Split principal/interest/penalty, overdue, allocation order.
UC10	Reversal/Refund	Đảo/hoàn giao dịch.	Không xóa ledger; ghi bút toán đảo/adjustment.
UC11	EOD/COB	Đóng ngày giao dịch.	Pending check, interest accrual, GL, reports, reconciliation.

UC01 - Balance Inquiry

Flow: User mở app -> MB Backend validate session -> ESB/core query account -> trả ledger/available balance -> app hiển thị.

Điểm cần chú ý: không dùng replica cũ cho quyết định giao dịch; nếu chỉ hiển thị có thể dùng cache/read model nhưng phải ghi rõ độ trễ. Với ATM/card, balance inquiry có thể đi qua ISO 8583.

UC02 - Internal Transfer

Flow: Validate user, OTP, from/to account, balance, limit -> idempotency -> lock -> debit/credit -> journal -> commit -> notification.

Rủi ro: timeout sau khi core commit, user retry, double debit. Cách xử lý là lưu requestId, query status, không retry mù.

UC03 - Interbank Transfer

Flow: Debit customer hoặc hold amount -> gửi instruction qua payment network/switch -> nhận response -> settlement/reconciliation.

Rủi ro: trạng thái có thể PENDING/UNKNOWN vì network không phản hồi ngay. Cần transaction state machine, query status, reconciliation file và exception handling.

UC04 - Merchant QR Payment

Flow: Merchant create order -> Payment Gateway tạo QR -> customer scan/pay -> bank/core authorize -> gateway callback merchant -> merchant update order.

Rủi ro: callback giả, callback trùng, merchant update PAID sai amount/currency/orderId, gateway success nhưng merchant timeout. Cần signature, idempotency, query status, reconciliation.

UC05 - ATM Withdrawal

Flow: ATM gửi authorization -> switch/NAPAS -> issuer/card/core debit hoặc hold -> ATM dispense cash -> nếu dispense fail thì reversal.

Rủi ro: account bị trừ nhưng ATM không nhả tiền. Bắt buộc có reversal/adjustment và đối soát ATM cassette/switch/core.

UC06/07 - Account & Term Deposit Opening

Flow: CIF/KYC -> chọn product -> tạo arrangement/account -> gán lãi/phí/GL -> nạp tiền ban đầu nếu có -> trả account/contract.

Rủi ro: product config sai dẫn đến lãi/phí sai hàng loạt; cần test rule, approval/maker-checker cho config quan trọng.

UC08/09 - Loan Disbursement & Repayment

Flow giải ngân: tạo loan contract -> posting loan asset debit, customer deposit credit -> schedule.

Flow trả nợ: debit customer deposit -> allocate principal/interest/penalty -> update outstanding.

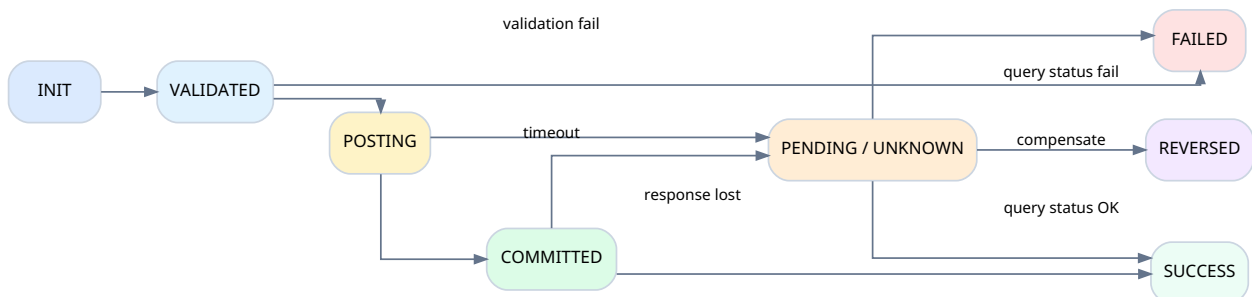
Rủi ro: allocation order sai, tính lãi sai, overdue/NPL sai, GL/P&L lệch.

UC10/11 - Reversal, Refund, EOD/COB

Không xóa dữ liệu giao dịch đã post. Nếu cần sửa ảnh hưởng tài chính, tạo reversal hoặc adjustment có audit. EOD/COB cần kiểm tra pending, lãi/phí, GL, reconciliation và report trước khi mở ngày mới.

10. Reliability patterns: idempotency, timeout, reversal, outbox

Sơ đồ 9 - State machine giao dịch tài chính



Một giao dịch banking nên có nhiều trạng thái hơn SUCCESS/FAILED để xử lý timeout, pending, reversal và compensation.

10.1 Idempotency key

Idempotency key là khóa đảm bảo cùng một request chỉ được xử lý một lần. Trong banking, nó thường là requestId/channelRef/orderId kết hợp channel/service.

```
ALTER TABLE transaction_log
ADD CONSTRAINT uk_transaction_request UNIQUE (channel, request_id);
```

Pseudo flow:

1. Receive requestId
2. Check existing transaction by channel + requestId
3. If exists: return previous result or query status
4. If not exists: create transaction INIT and process
5. Mark SUCCESS/FAILED/PENDING with coreRef

10.2 Timeout handling

Timeout không đồng nghĩa thất bại. Sau khi gửi request vào core/payment network, hệ thống gọi có thể timeout trong khi core đã commit. Vì vậy tuyệt đối không retry mà giao dịch tiền.

1. Nếu timeout trước khi chắc chắn request đã tới core: retry được nếu có idempotency.
2. Nếu core đã nhận request hoặc đã có coreRef: gọi query status.
3. Nếu query status SUCCESS: trả success/cập nhật trạng thái.
4. Nếu query status FAILED: trả lỗi hoặc cho retry có kiểm soát.
5. Nếu UNKNOWN lâu: đưa vào pending queue/reconciliation.
6. Nếu cần hủy ảnh hưởng tài chính đã ghi: dùng reversal/compensation, không xóa dữ liệu.

10.3 Retry có kiểm soát

Lỗi	Có retry không?	Ghi chú
Network timeout trước khi tới backend	Có, nếu có idempotency.	Retry an toàn hơn nếu requestId không đổi.
Core timeout sau khi gửi request	Không retry mù.	Phải query status.
Validation failed	Không.	Account closed, insufficient funds, invalid OTP.
Database deadlock	Có thể retry ngắn.	Cần backoff và idempotency.
Duplicate request	Không posting lại.	Trả kết quả cũ hoặc trạng thái hiện tại.
Payment network unavailable	Tùy flow.	Có thể pending hoặc reject trước khi debit.

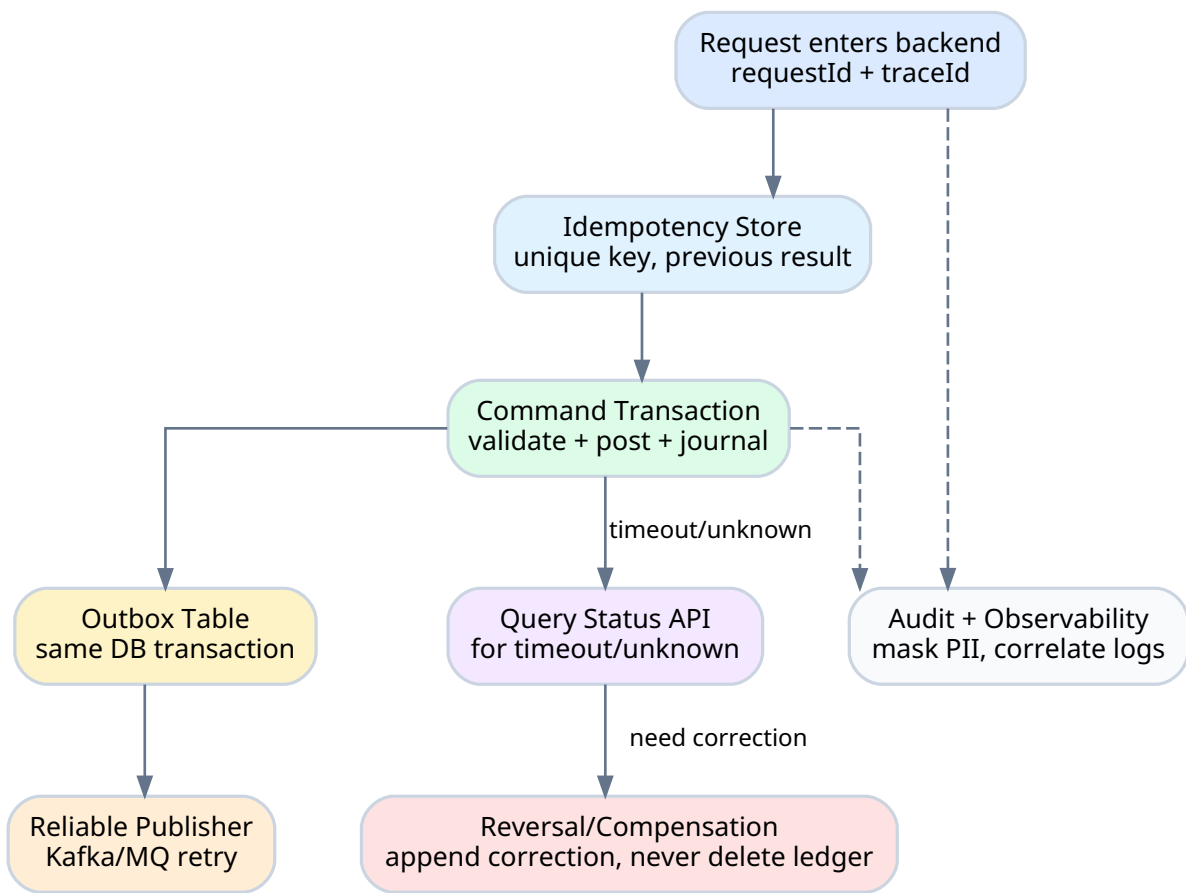
10.4 Reversal vs Refund vs Adjustment

Khái niệm	Dùng khi nào	Ví dụ
Reversal	Đảo giao dịch khi giao dịch ban đầu cần hủy/ không hoàn tất đúng.	ATM debit thành công nhưng ATM không nhả tiền -> reversal trả tiền.
Refund	Hoàn tiền sau giao dịch merchant payment đã thành công.	Khách mua hàng online rồi trả hàng -> merchant refund.
Adjustment	Điều chỉnh có kiểm soát khi có lệch cần sửa.	Đối soát phát hiện thiếu/ thừa settlement.

10.5 Outbox pattern

Outbox pattern giúp đảm bảo DB commit và event publish không bị lệch. Trong cùng DB transaction, hệ thống update transaction SUCCESS và insert outbox_event. Worker riêng đọc outbox_event để publish Kafka/MQ, rồi mark published.

Sơ đồ 10 - Safe integration patterns



Kết hợp idempotency, command transaction, outbox, query status, reversal và audit để tích hợp an toàn.

10.6 Circuit breaker, bulkhead, timeout budget

- **Circuit breaker:** nếu core/payment network lỗi nhiều, tạm ngắt gọi để tránh cascade failure.
- **Bulkhead:** tách thread pool/connection pool theo service để một service lỗi không kéo sập toàn bộ backend.
- **Timeout budget:** phân bổ timeout theo layer. Không để Gateway timeout 30s, backend timeout 35s, core timeout 60s gây request treo và retry rối loạn.

11. Data architecture: OLTP, ledger, DWH, locking, audit

11.1 OLTP vs Reporting/DWH

Core Banking là OLTP system: xử lý transaction nhanh, chính xác, consistent. Reporting/DWH là analytical system: đọc nhiều, aggregate nhiều, phục vụ báo cáo. Không nên chạy báo cáo nặng trực tiếp trên bảng transaction nóng của core.

```
Core OLTP DB
-> CDC / Batch / ETL
-> ODS / Data Warehouse / Data Mart
-> BI / MIS / Regulatory Reports
```

11.2 Locking trong giao dịch tiền

Khi hai giao dịch cùng trừ một account, phải tránh race condition. Ví dụ account A có 1,000,000 VND, hai request cùng chuyển 800,000 VND. Nếu cả hai đọc balance trước khi update, cả hai đều tưởng đủ tiền.

Cách xử lý	Khi dùng	Lưu ý
Pessimistic lock	Giao dịch tiền cần serialize trên account/balance row.	SELECT ... FOR UPDATE, tránh giữ lock quá lâu.
Optimistic lock	Concurrency thấp hoặc update theo version.	Cần retry khi version conflict.
Append-only ledger + projection	Hệ thống ledger hiện đại/event sourcing.	Phải có projection consistency rõ ràng.
Partition by account	Event-driven/stream processing.	Đảm bảo ordering theo account key.

11.3 Append-only ledger

Nhiều hệ thống tài chính dùng append-only ledger: không sửa/xóa journal line đã ghi. Nếu sai thì ghi bút toán đảo hoặc adjustment. Lợi ích là audit tốt, dễ rebuild balance và dễ điều tra sự cố.

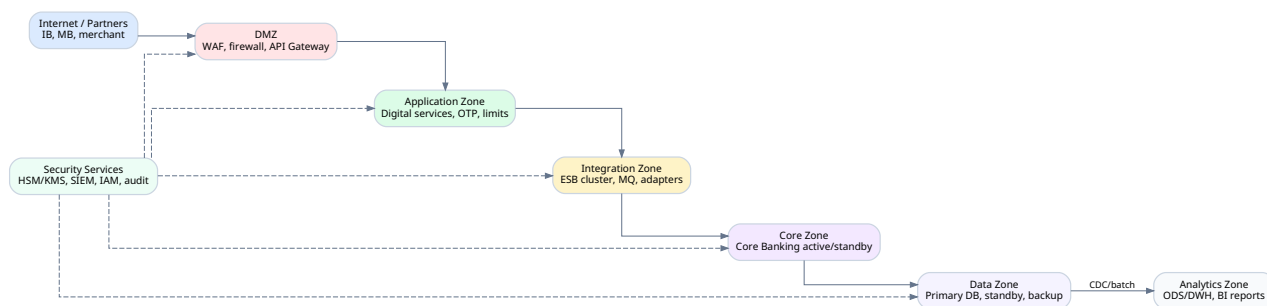
```
Giao dịch sai:
Debit  Account A  1,000,000
Credit Account B  1,000,000

Không xóa dòng cũ.
Tạo reversal:
Debit  Account B  1,000,000
Credit Account A  1,000,000
```

11.4 Primary/Standby DB và replica lag

Trong sơ đồ training có Oracle Master/Slave. Có thể hiểu đơn giản: Primary nhận write transaction; Standby/Replica replicate dữ liệu để read/reporting/DR. Với banking, replica lag rất nhạy cảm. Không dùng dữ liệu replica cũ cho quyết định như available balance nếu chưa đảm bảo consistency.

Sơ đồ 11 - Deployment architecture tham khảo



Phân vùng Internet/DMZ/App/Integration/Core/Data/Analytics và security services như HSM, SIEM, IAM.

11.5 Audit log cần lưu gì?

Audit field	Mục đích
trace_id / correlation_id	Trace xuyên API Gateway, backend, ESB, core.
actor / user_id / operator_id	Ai thực hiện thao tác.
channel / device_id / ip	Từ kênh nào, thiết bị nào, địa chỉ nào.
service_code / request_id / core_ref	Liên kết nghiệp vụ và core transaction.
masked request/response	Phục vụ điều tra nhưng không lộ PII/PAN/OTP/PIN.
before/after status	Biết trạng thái thay đổi ra sao.
timestamp theo business date và system time	Phân biệt ngày giao dịch và thời gian hệ thống.

12. Security architecture: mTLS, HSM, masking, maker-checker

12.1 Các lớp bảo mật trong banking

Lớp	Ví dụ	Mục tiêu
Network security	Firewall, WAF, subnet, DMZ, security zone.	Giảm bề mặt tấn công.
Transport security	TLS, mTLS, certificate pinning nếu cần.	Bảo vệ dữ liệu trên đường truyền và xác thực service.
Application security	OAuth2/OIDC, session, RBAC, maker-checker.	Đảm bảo đúng user/quyền/kênh.
Data security	Encryption at rest, encryption in transit, masking.	Bảo vệ dữ liệu nhạy cảm.
Key management	HSM, KMS, key rotation.	Bảo vệ key, PIN, MAC, signature.
Audit/SIEM	Immutable audit log, alert, anomaly detection.	Phát hiện và điều tra sự cố.
Fraud/Risk	Device fingerprint, velocity check, anomaly detection.	Giảm gian lận giao dịch.
Compliance	KYC, AML, retention, access control.	Tuân thủ quy định và nội bộ.

12.2 HSM là gì?

HSM là Hardware Security Module. Trong banking/payment, HSM dùng để bảo vệ key và thực hiện crypto operation nhạy cảm như PIN block, MAC/signature verification, card key management, secure key storage. Ý tưởng là key nhạy cảm không nằm lộ trong database hoặc application memory bình thường.

12.3 Masking và PII

Không được log dữ liệu nhạy cảm đầy đủ. Điều này rất quan trọng khi backend log raw request/response từ payment/core.

Dữ liệu	Cách log an toàn
Account number	001123*****789
Card/PAN	9704 12** **** 1234
Phone	090****789
Email	n***@domain.com
OTP/PIN/CVV/Password	Không log. Không lưu plain text.
Full request có PII	Chỉ log masked fields hoặc lưu vào vault/audit store có quyền hạn chế.

12.4 Maker-checker

Maker-checker là pattern phổ biến: người tạo giao dịch/cấu hình không phải là người duyệt. Ví dụ nhân viên A tạo lệnh chuyển tiền doanh nghiệp 10 tỷ, nhân viên B kiểm tra, manager C duyệt; core chỉ post sau khi được authorize.

Khi nào cần maker-checker?

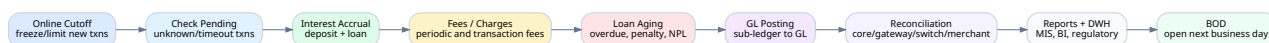
Giao dịch giá trị lớn, giao dịch doanh nghiệp, thay đổi product config, thay đổi lãi/phí/GL mapping, thao tác vận hành nhạy cảm như reversal/adjustment/manual posting.

13. Batch/EOD, reconciliation và vận hành production

13.1 EOD/COB là gì?

Core Banking không chỉ chạy online real-time. EOD/COB là quá trình đóng ngày giao dịch: kiểm tra transaction trong ngày, tính lãi/phí, xử lý loan overdue, tổng hợp GL, đối soát, sinh report và mở ngày mới.

Sơ đồ 12 - Batch/EOD flow



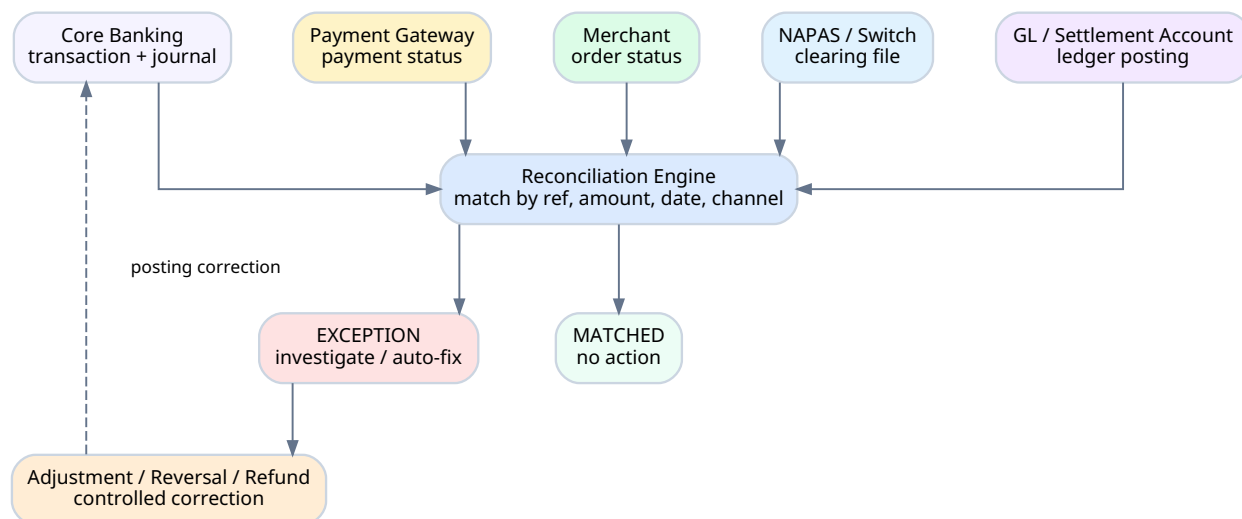
EOD thường đi qua cutoff, pending check, interest accrual, fees, loan aging, GL posting, reconciliation, reports và BOD.

Batch	Ý nghĩa
BOD - Beginning of Day	Mở ngày giao dịch mới.
EOD - End of Day	Đóng ngày giao dịch, tổng hợp, kiểm tra cân bằng.
Interest Accrual	Tính lãi phát sinh hằng ngày.
Interest Capitalization	Nhập lãi vào gốc/tài khoản.
Fee Charging	Thu phí định kỳ hoặc phí theo giao dịch.
GL Posting	Tổng hợp hạch toán sang GL.
Statement Generation	Sinh sao kê.
Reconciliation	Đối soát core, gateway, switch, merchant, GL.
Report MIS/BI	Báo cáo quản trị, vận hành, regulatory.
DWH Sync	Đồng bộ dữ liệu sang data warehouse.

13.2 Reconciliation là gì và vì sao quan trọng?

Reconciliation là đối soát dữ liệu giữa các hệ thống để phát hiện lệch: merchant nói PAID, gateway nói SUCCESS, bank/core có posting hay không, NAPAS/network file có dòng không, GL/settlement account có cân không.

Sơ đồ 13 - Reconciliation flow



Reconciliation engine lấy dữ liệu từ core, gateway, merchant, NAPAS/switch và GL để match hoặc tạo exception.

13.3 Production runbook nên có gì?

- Dashboard trạng thái core API, ESB, payment gateway, MQ, DB, batch jobs.
- Alert cho timeout tăng, pending tăng, duplicate tăng, deadlock tăng, GL imbalance.
- Runbook xử lý transaction UNKNOWN: query status, wait, reconcile, reversal nếu cần.
- Runbook EOD fail: job nào fail, có thể rerun không, ảnh hưởng GL/balance không.
- Quy trình manual adjustment/reversal có maker-checker và audit.
- Backup/restore test, DR drill, capacity test, security test định kỳ.

14. Checklist thiết kế API banking cho backend

Trước khi thiết kế hoặc review một API banking, hãy dùng checklist này. Nó giúp bạn hỏi đúng câu hỏi trước khi code.

14.1 Checklist nghiệp vụ

- Nghiệp vụ này có làm thay đổi tiền không?
- Cần debit/credit account/GL nào?
- Dùng ledger balance hay available balance?
- Có hold/freeze/block/minimum balance không?
- Có daily limit/transaction limit/product limit không?
- Có cần OTP/2FA/maker-checker không?
- Có phí, VAT, tax, exchange rate không?
- Có ảnh hưởng batch EOD/interest/GL không?

14.2 Checklist kỹ thuật

- Idempotency key là gì?
- Có lưu transaction trước khi gọi core không?
- Timeout xử lý ra sao?
- Có query status API không?
- Có reversal/refund/compensation không?
- Có lock balance/account đúng cách không?
- Có audit log và traceId không?
- Log đã mask PII/PAN/OTP chưa?
- Downstream event có dùng outbox không?
- Monitoring/alert nào cần có?

14.3 API contract ví dụ: chuyển tiền nội bộ

```
POST /api/v1/transfers/internal
Idempotency-Key: MB-20260630-000001
X-Trace-Id: 6f0bfa7a9a7d4f1d
Authorization: Bearer <token>
```

```
{
  "fromAccount": "00110001111",
  "toAccount": "00110002222",
  "amount": 1000000,
  "currency": "VND",
  "remark": "Dinner",
  "otpRef": "OTP-123456"
}
```


14.4 Response status nên thiết kế

HTTP	Business status	Ý nghĩa
200	SUCCESS	Giao dịch thành công, đã có coreRef/posting.
202	PENDING	Chưa rõ kết quả, cần query status/reconciliation.
400	VALIDATION_FAILED	Request sai hoặc vi phạm rule đầu vào.
409	DUPLICATE_REQUEST	RequestId đã xử lý hoặc đang xử lý.
422	INSUFFICIENT_FUNDS	Không đủ available balance.
500	SYSTEM_ERROR	Lỗi hệ thống; nếu đã gửi core phải giữ pending/unknown.
504	TIMEOUT	Timeout; không retry mù, cần query status.

15. Glossary thuật ngữ Core Banking

Thuật ngữ	Giải thích rõ ràng
Account	Tài khoản dùng để ghi nhận số dư hoặc nghĩa vụ tài chính.
Accounting	Hạch toán kế toán, ghi debit/credit vào journal/ledger.
Acquirer	Bên/ngân hàng chấp nhận giao dịch thẻ tại ATM/POS/merchant.
API Gateway	Lớp expose API, auth, rate limit, routing, monitoring.
Arrangement	Trong T24 AA, hợp đồng/sản phẩm cụ thể của khách hàng được tạo từ product.
ATM	Automated Teller Machine, máy rút tiền/giao dịch tự động.
Authorization	Bước kiểm tra/chấp nhận giao dịch online.
Available Balance	Số dư có thể sử dụng sau khi trừ hold/block/minimum balance.
BOD	Beginning of Day, mở ngày giao dịch.
CASA	Current Account/Savings Account, tài khoản thanh toán/tiết kiệm không kỳ hạn.
CIF	Customer Information File, hồ sơ định danh khách hàng.
Clearing	Trao đổi/xác nhận dữ liệu giao dịch giữa các bên sau giao dịch.
COB	Close of Business, quy trình đóng ngày kinh doanh.
Core Banking	Hệ thống lõi xử lý account, balance, product, transaction, accounting.
Credit	Bên có trong accounting; tăng liability/equity/income.
Debit	Bên nợ trong accounting; tăng asset/expense.
Deposit	Module tiền gửi: account, balance, interest, transfer.
DWH	Data Warehouse, kho dữ liệu báo cáo/phân tích.
EOD	End of Day, đóng ngày giao dịch.
ESB	Enterprise Service Bus, tầng tích hợp route/transform/protocol conversion.
Fixed-length message	Message có field độ dài cố định, phổ biến trong legacy core.
FX	Foreign Exchange, nghiệp vụ ngoại hối.
GL	General Ledger, sổ cái tổng hợp.
Hold Amount	Số tiền bị giữ tạm, chưa cho dùng.
HSM	Hardware Security Module, thiết bị bảo vệ key/crypto operation.
IB	Internet Banking, kênh web banking.
Idempotency	Cơ chế đảm bảo cùng request không bị xử lý nhiều lần.
ISO 8583	Chuẩn message cho giao dịch thẻ/acquirer/issuer.
ISO 20022	Chuẩn message tài chính hiện đại cho payment/settlement.
Issuer	Bên/ngân hàng phát hành thẻ/tài khoản của khách hàng.
Journal	Sổ nhật ký ghi transaction theo thời gian với debit/credit lines.
Ledger Balance	Số dư đã ghi sổ.
Loan	Module khoản vay: gốc, lãi, lịch trả nợ, dư nợ.

Thuật ngữ	Giải thích rõ ràng
MB	Mobile Banking, kênh app banking.
NAPAS	Hệ thống chuyển mạch tài chính tại Việt Nam.
Nostro	Tài khoản ngân hàng mở tại ngân hàng khác, phục vụ thanh toán.
OFS	Open Financial Service, kiểu message/interface truyền thống tích hợp T24.
Outbox Pattern	Pattern lưu event vào DB cùng transaction rồi publish sau để tránh mất event.
Payment Gateway	Cổng thanh toán kết nối merchant với bank/network/payment method.
Posting	Biến giao dịch nghiệp vụ thành bút toán debit/credit.
Product	Sản phẩm ngân hàng kèm rule lãi, phí, limit, GL mapping.
Reconciliation	Đối soát dữ liệu giữa các hệ thống.
Refund	Hoàn tiền sau giao dịch đã thành công.
Reversal	Đảo giao dịch để sửa/hủy ảnh hưởng tài chính của giao dịch trước.
Settlement	Quyết toán tiền thật giữa các bên.
Treasury	Quản lý nguồn vốn, thanh khoản, cash position.
VNPAY	Fintech/payment provider tại Việt Nam.
WAF	Web Application Firewall.

16. Phụ lục Mermaid source và nguồn tham khảo

16.1 Mermaid source

Các sơ đồ trong tài liệu được render dưới dạng hình để PDF ổn định. Phần dưới đây là Mermaid source tương ứng để bạn copy sang Mermaid Live Editor, Markdown, Obsidian hoặc documentation nội bộ.

01 - Architecture layers

```
flowchart TD
    A[Channel Layer<br/>IB/MB/Web, ATM, POS, Card, Branch, Partner]
    B[Security / Edge<br/>WAF, Firewall, Load Balancer, TLS, DMZ]
    C[API Gateway<br/>Auth, rate limit, routing, observability]
    D[Digital Domain<br/>IB/MB service, session, OTP, limit, device risk]
    E[ESB / Integration Layer<br/>Route, transform, protocol adapter, MQ, audit]
    F[Core Banking<br/>CIF, Account, Deposit, Loan, Card, FX, Treasury, GL]
    G[Data Layer<br/>DB, Journal, Ledger, Balance, Audit, DWH]
    H[External Networks<br/>NAPAS, VNPAY, SMS, AML/Fraud, CRM]
    A --> B --> C --> D --> E --> F --> G
    E --> H
    G -. report/recon .-> H
```

02 - Core domain chain

```
flowchart LR
    CIF[Customer/CIF] --> ACC[Account]
    PROD[Product] --> ACC
    ACC --> TXN[Transaction]
    TXN --> POST[Posting<br/>Debit/Credit]
    POST --> J[Journal]
    J --> GL[General Ledger]
    GL --> R[Report/Reconciliation]
```

03 - Channel to core sequence

```
sequenceDiagram
    participant U as IB/MB/Card Client
    participant G as API Gateway
    participant D as Digital Backend
    participant E as ESB/Adapter
    participant C as Core Banking
    participant DB as DB/Ledger
    U->>G: HTTPS JSON request
    G->>D: Authenticated request
    D->>D: Validate session, OTP, limit
    D->>E: Canonical banking message
    E->>C: Fixed-length/OFS/ISO/core message
    C->>DB: Validate, lock, posting, journal
    DB-->>C: Commit + response code
    C-->>E: Core response
    E-->>D: Normalized response
    D-->>U: JSON result
```

04 - Internal transfer sequence

```
sequenceDiagram
    participant MB as Mobile Backend
    participant Store as Idempotency Store
    participant Core as Core Deposit
    participant DB as Ledger DB
    participant Outbox as Outbox/Kafka
    MB->>Store: check requestId
    Store-->>MB: not found
    MB->>Core: transfer A -> B
    Core->>DB: lock account A/B
    Core->>DB: debit A, credit B, journal lines
    DB-->>Core: commit
    Core-->>MB: SUCCESS + coreRef
    MB->>Outbox: publish TransferSuccessEvent asynchronously
```

05 - Payment gateway flow

```
sequenceDiagram
    participant C as Customer
    participant M as Merchant
    participant PG as Payment Gateway
    participant N as Bank/Switch/NAPAS
    participant Core as Core/Card System
    C->>M: checkout order
    M->>PG: create payment request
    PG-->>C: payment URL/QR
    C->>N: authenticate and pay
    N->>Core: authorize/post/hold
    Core-->>N: result
    N-->>PG: payment result
    PG-->>M: signed callback/webhook
    M->>M: update order status
```

06 - Transaction state machine

```
stateDiagram-v2
    [*] --> INIT
    INIT --> VALIDATED
    VALIDATED --> POSTING
    POSTING --> COMMITTED
    COMMITTED --> SUCCESS
    VALIDATED --> FAILED: validation fail
    POSTING --> PENDING: timeout
    COMMITTED --> PENDING: response lost
    PENDING --> SUCCESS: query status OK
    PENDING --> FAILED: query status failed
    PENDING --> REVERSED: compensation/reversal
```

07 - Reconciliation flow

```
flowchart TD
    A[Core transaction/journal]
    B[Gateway payment status]
    C[Merchant order status]
    D[NAPAS/switch clearing file]
```

```

E[GL/settlement account]
F[Recon engine<br/>match by ref, amount, date, channel]
G[MATCHED]
H[EXCEPTION]
I[Adjustment/Reversal/Refund]
A --> F
B --> F
C --> F
D --> F
E --> F
F --> G
F --> H --> I
I -. correction posting .-> A

```

08 - EOD flow

```

flowchart LR
  A[Online cutoff] --> B[Check pending/unknown]
  B --> C[Interest accrual]
  C --> D[Fees/charges]
  D --> E[Loan aging/penalty]
  E --> F[GL posting]
  F --> G[Reconciliation]
  G --> H[Reports + DWH]
  H --> I[BOD - next business day]

```

09 - Logical data model

```

erDiagram
  CUSTOMER ||--o{ ACCOUNT : owns
  PRODUCT ||--o{ ACCOUNT : configures
  ACCOUNT ||--|| BALANCE : has
  ACCOUNT ||--o{ TRANSACTION : participates
  TRANSACTION ||--o{ JOURNAL_ENTRY : creates
  JOURNAL_ENTRY ||--o{ JOURNAL_LINE : contains
  LEDGER_ACCOUNT ||--o{ JOURNAL_LINE : posted_to
  TRANSACTION ||--o{ AUDIT_LOG : traced_by

```

10 - Safe integration patterns

```

flowchart TD
  A[RequestId + TraceId] --> B[Idempotency Store]
  B --> C[Command transaction<br/>validate + post + journal]
  C --> D[Outbox table<br/>same DB transaction]
  D --> E[Reliable event publisher]
  C --> F[Query status API<br/>timeout/unknown]
  F --> G[Reversal/compensation<br/>append correction]
  C -. H[Audit + Observability<br/>mask PII]

```

16.2 Nguồn tham khảo

- [S1] Temenos Developer - Transact APIs / Arrangement Architecture. <https://developer.temenos.com/transact-apis>
- [S2] IBM - What Is an Enterprise Service Bus (ESB)? <https://www.ibm.com/think/topics/esb>
- [S3] Google Cloud - About API Gateway. <https://docs.cloud.google.com/api-gateway/docs/about-api-gateway>

4. [S4] ISO - ISO 8583 Financial transaction card originated messages. <https://www.iso.org/standard/31628.html>
5. [S5] ISO 20022 - Universal financial industry message scheme. <https://www.iso20022.org/iso-20022>
6. [S6] NAPAS - ATM/POS Domestic Switching Service. <https://en.napas.com.vn/atmpos-domestic-switching-service>
7. [S7] VNPAY - Payment ecosystem. <https://vnpay.vn/>
8. [S8] OpenLearn - Accounting equation and double-entry rules. <https://www.open.edu/openlearn/money-business/introduction-bookkeeping-and-accounting/content-section-3.6>
9. [S9] Thought Machine - Vault Core. <https://www.thoughtmachine.net/vault-core>
10. [S10] Oracle - FLEXCUBE Universal Banking. <https://www.oracle.com/financial-services/banking/flexcube/core-banking-software/>

16.3 Tóm tắt một trang

Nhớ nhanh

Architecture: Channel -> API Gateway -> Digital Domain -> ESB -> Core -> DB -> Report/External Network.

Domain: Customer -> Account -> Product -> Transaction -> Posting -> Journal -> Ledger -> Report.

Backend safety: Idempotency, timeout handling, query status, reversal/refund, locking, audit, reconciliation, security/masking.

Accounting: Every financial transaction must balance: total debit = total credit.